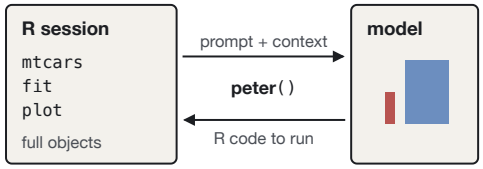
Agents in your R session with gptr :: CHEATSHEET

`peter()` runs language model agents in your current R session, on objects already in memory. The name honors Peter Naur's work on readable language definitions and Peter Cathcart Wason's research with Jonathan Evans on reasoning. In Naur's spirit, gptr records sessions as runnable documents.

How it works



Objects stay in R; the agent's code runs there on the full data. An agent is sent the prompt, bounded previews, a workspace listing, the R code you ran since the last prompt, project instructions and tool output (`?gptr_egress`; `.opts = list(context = "none")` drops listing and code); a decision model, each input.

gptr_describe(`mtcars`)
What a model is shown of an object.

Set up

pak::pak(`"Broccolito/gptr@main"`)
library(`gptr`)
Install gptr 1.0 from GitHub (R 4.2 or later).

gptr_env(`"~/keys/.env"`)
Load keys from a private .env file; gptr redacts them.

gptr_login(`"openrouter"`)
Sign in interactively: OAuth for OpenRouter, masked key entry for other providers.

gptr_providers()
Each route's credential, status, default model, egress.

gptr_models(`"sonnet"`)
Search the offline catalog by name, alias or provider.

gptr_config(`model = opus, .scope = "user"`)
Save a default model. Scopes: session, project, user.

gptr_init(`"."`)
Create a `.gptr/` workspace so sessions and recorded answers outlive R (else they end with it); at the console it then asks whether to trust the project.

gptr_trust(`".", trust = TRUE`)
Trust without the question, as in a script, or change it.

gptr_permissions(`allow = "r(fn:saveRDS)"`)
Add a rule for this R session; no arguments lists rules.

`fake = `**gptr_fake_provider**(`list("32 rows."))`
A scripted offline model for examples and tests.

No model set? Interactive `peter()` (no prompt) offers the Codex CLI, the Claude Code CLI or manual API or Ollama setup, and saves a CLI choice as default. `peter("...")` takes the first ready route instead (an Anthropic, OpenAI or Google key, else Claude Code, else Codex) and asks once before sending context.

Ask Peter from R

`peter("...")` returns a session, in scripts, loops and functions too. Where nobody can approve (Rscript, knitr), pass `mode = auto` or an allow rule, or the first change stops the run (`gptr_error_permission`).

`s = `**peter**`("Fit mpg on wt; save it as fit.", mtcars)`
The prompt first, then the objects to work on.

s$text; s$value; s$turns
Last answer, newest `gptr_return()`, prompt count.

s$usage; s$cost; s$envir
Usage, cost (NA without prices) and workspace (NULL in a function unless `envir = new.env()`).

`col = "mpg"; `**peter**`("Plot {col} by cyl.")`
Literal prompts interpolate `{name}` from R.

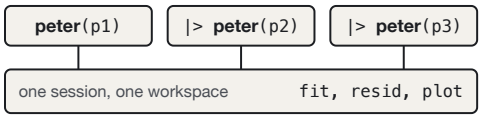
peter`("Clean the data.", raw, mode = auto, model = "openai/gpt-6-sol")`
Name other arguments exactly; bare names work.

peter`("Summarise.", sales, budget = list(tokens = 5e4, turns = 5))`
Per-call limits, checked before each request; `turns` counts model requests, `cost` needs priced models.

gptr_return`(coef(fit))  # in agent R code`
Marks the result, `s$value` until a later one replaces it.

tryCatch`(`**peter**`("Fit.", d), gptr_error = function(e) e$session)`
A stopped run carries `e$session` (after Esc or Ctrl-C, `gptr_last()`); `gptr_rewind(e$session)` drops its failed turn. A call refused up front changes nothing.

Pipe: one session, many turns



`s |> `**peter**`("Now plot the residuals.")`
Same session and workspace, one more turn.

peter`("Draft a summary.", fit) |> `**peter**`("Shorten it.")`
Each pipe adds a turn to the session on its left.

`b = `**gptr_fork**`(s)`
`b |> `**peter**`("Try a log model instead.")`
Branch off; only in-place edits (data.table `:=`, environments, `<<-`) reach the original's objects.

`s |> `**peter**`("Add a caveat.", model = opus)`
Switch the model mid-pipe; the history is kept.

`s; `**summary**`(s); `**gptr_last**`()`
Print it, its history, the most recent session.

The console

`session = `**peter**`()`
`session |> `**peter**`()`
Open the console; `/exit` returns the session.

AT THE peter> PROMPT

`Fit mpg on wt.`	A prompt for the session
`!head(d)`	Run R and show the model
`!!x = 2`	Run R; only object changes are sent
```` ```r … ``` ````	Fenced R, run like `!code`
`""" … """`	A multi-line prompt
`line \`	Continue on the next line
`@notes.md`	Attach a file's first 40 lines
`@fit`	Attach an object
`{n}`	Insert an R value

SLASH COMMANDS

`/help [cmd]`	Commands and syntax
`/model [ref]`	Show or switch model
`/mode [mode]`	Show or switch mode
`/status`	Session, model, mode
`/cost /context`	Tokens and cost
`/env [text]`	Workspace objects
`/tools`	Tools and helpers
`/compact [focus]`	Compact the history
`/clear`	New chat, same objects
`/resume [id]`	List or resume sessions (across R restarts only in a `.gptr/` project)
`/fork`	Fork and continue there
`/doc [path]`	The history document
`/skills /skill:name`	List or use skills
`/mcp`	List MCP servers
`/permissions`	Show or change rules
`/undo /redo /retry`	Undo, redo, resend
`/rewind [k]`	Keep turns 1 to k
`/exit`	Leave; return the session (NULL if none yet)

WHILE AN ANSWER RUNS

In a terminal (not Windows), type, then Enter steers and Tab follows up. Ctrl-C (RStudio: Esc) pauses: **[s]**teer, **[f]**ollow-up, **[c]**ontinue, **[a]**bort, **[b]**ackground (in the console it runs on only after `/exit` or `!!gptr_wait(gptr_last())`). Approvals (same key aborts): **[y]**es, **[a]**lways (session), **[p]** always (project), **[n]**o (`n <text>` says what to do instead), **[?]**.

System 1 and System 2

System 1 typed decision → `TRUE "lung" 1.88` vector + probabilities

System 2 agent with tools → `gptr_session` $text, $value, objects

The model decides what `peter()` returns. A decision model (Jev, Clef) answers once per input; a conversational model works in turns.

SYSTEM 1: TYPED DECISIONS

`rct = `**peter**`("Is this an RCT?", abstracts, model = jev)`
A logical vector: TRUE when P(yes) ≥ `threshold` (0.5).

gptr_prob`(rct)`
P(yes) per element; `"probabilities"` gives the matrix.

peter`("Which tissue?", cells, model = jev, choices = c("liver", "lung", "other"))`
One label per input: a character vector.

peter`("How severe?", events, model = jev, levels = c("low", "mid", "high"))`
A score: the expected zero-based level, such as 1.88.

peter`(q, x, model = jev, min_confidence = 0.8, uncertain = "stop")`
Abstain below a confidence: for yes/no |2p−1| (0.8 keeps p ≤ 0.1 or ≥ 0.9), for choices and scores the engine's own scale. NA by default; `TRUE/FALSE` (yes/no) or `function(state, answer)` fills in; `"stop"` errors, naming the inputs.

`abstracts[rct]; `**ifelse**`(rct, "RCT", "other")`
Plain vectors for `if`, `for`, subsetting and `ifelse()`.

`s |> `**peter**`("Did it succeed?", model = jev)`
Judge a session's last answer; adds no turn.

`m = if (isTRUE(hard)) "opus" else "haiku"`
peter`(task, model = m)`
Route each task to a deep or a fast agent.

SYSTEM 2: AGENTS

peter`("Plan it.", d, model = opus, .opts = list(thinking = "high"))`
More reasoning: `off` to `max`, or a model suffix such as `:high`; a model without that level uses its nearest one and says so (opus has no `off`).

Try it offline

```
step = list(tool = "r", input = list(
  code = "fit = lm(mpg ~ wt, mtcars)"))
fake = gptr_fake_provider(list(step, "Done."))
s = peter("Fit mpg on wt.", mtcars,
  model = fake, mode = auto); coef(fit)
```

No key: the first reply runs R, the second is `s$text`.

Models and providers

```
"anthropic/claude-sonnet-5-5:high"
```
provider · model id · thinking

Aliases work too: `sonnet`, `opus`, `haiku`, `gpt`, `flash`, `jev`, `claude_code`, `codex`.

`anthropic/…`	Anthropic `ANTHROPIC_API_KEY`
`openai/…`	OpenAI `OPENAI_API_KEY`
`google/…`	Google Gemini `GEMINI_API_KEY`
`openrouter/…`	OpenRouter; also groq, deepseek, mistral, together, xai, cerebras and fireworks routes, listed after `gptr_models(refresh = TRUE)` <NAME>_API_KEY
`azure/…`	Azure OpenAI `AZURE_OPENAI_API_KEY`, `AZURE_OPENAI_ENDPOINT`
`bedrock/…`	AWS Bedrock `AWS_BEARER_TOKEN_BEDROCK`, `AWS_REGION`
`ollama/…`	Ollama, local, no key. Also lmstudio, llamacpp, vllm: tools once a model is declared with `tool_call = TRUE`
`claude-cli`	Claude Code CLI `claude auth login`
`codex`	Codex CLI `codex login`
`jev`	System 1 · TypeSafe Jev `TYPESAFE_API_KEY`
`ollama/clef`	System 1 · local Clef and Clef Flash, Ollama 0.35.1 or later

gptr_models(provider = "ollama",
  refresh = TRUE)
Ask Ollama which models are installed; for a cloud route, download the models.dev catalog of all routes.

```
m = list(id = "my-id", tool_call = TRUE)
gptr_config(providers = list(
  bedrock = list(models = list(m))))
```
Declare Bedrock or Azure models, then `model = "bedrock/my-id"`. Each call replaces `providers`.

gptr_register(**gptr_provider**("corp",
  "openai-completions", base_url = u,
  auth = "CORP_KEY", models = list(m)))
Any OpenAI-compatible endpoint; registering the name again replaces it. `api = "bedrock-converse"` with `compat = list(region = "us-west-2")` signs Bedrock with SigV4: `auth` names the secret key, `AWS_ACCESS_KEY_ID` holds the key id.

```
eg = gptr_config()$egress
eg$openai = "ack"
gptr_config(egress = eg, .scope = "user")
```
Acknowledge egress to openai, for unattended runs.

The script is the history

Bind the document with your `peter()` calls; each top-level call gets a block of the agent's recorded R code.

```
peter("Fit mpg on wt in mtcars.")
# >>> gptr:46d369 model=…
fit = lm(mpg ~ wt, mtcars)
coef(fit)[["wt"]]
#> [1] -5.344472
## Decision: linear fit
# <<< gptr:46d369
```
prompt · header · code · output · note · end

gptr_doc("analysis.R")
Record this R session's calls into the `.R`, `.Rmd`, `.qmd` or `.ipynb` holding them; `quarto render` and Knit start another R session, so bind in the setup chunk.

gptr_config(mode = auto, .scope = "session")
Under `Rscript` or `rmarkdown::render()` nobody can approve: record with auto mode (or the call's `mode =`) or `gptr_permissions()` rules; replays need none.

gptr_source("analysis.R",
  replay = "replay")
Rerun: fresh blocks replay with no model call; a new R session gets `s$text` back only from `.gptr/`.

```
GPTR_REPLAY=replay Rscript analysis.R
```
Prove from the shell that a script needs no model.

gptr_blocks("analysis.R")
Each block's status: fresh, stale (prompt edited), user-edited or undone.

gptr_doc("report.Rmd")
rmarkdown::render("report.Rmd")
Render as usual; blocks become `gptr-<id>` chunks.

gptr_doc("analysis.ipynb", sync = TRUE)
After closing the notebook, run this, in any R session, to write the blocks queued while it was open.

gptr_doc("log.R") # at the console
Append each console turn to a runnable `.R` transcript as `s_<id> = peter("…")`, with `!code` lines and slash commands. R typed at the > prompt is not recorded, so create inputs with `!code`.

peter(q, replay = "live")
Modes: `"auto"` (default), `"replay"`, `"live"`, `"record"`. `"live"` re-asks the model under `gptr_source()`, knitr, an IDE or Jupyter; plain `source()` and `Rscript` replay a recorded call instead, with a warning.

EXPORT A RECORDED .R

Export needs fresh blocks and `peter()` calls whose arguments are all quoted strings, numbers or TRUE/FALSE (write `mode = "auto"`): no data objects, `.opts` or {name} prompts; `s |> peter()` only continues an `s` an earlier `peter()` statement assigned.

gptr_export("analysis.R", "plain.R")
Plain R: recorded code runs, `peter()` calls become comments; later code reading `s` (`s$value`) is refused.

gptr_export("analysis.R", "peter.R",
  format = "peter")
`peter()` calls run; recorded code is commented.

Tools, files and artifacts

The model calls `read`, `r`, `edit`, `write` and `ask`; scripts offer `ask` only in `manual` mode, where it stops the run. Inside `r` it uses `peter$` helpers, which you can call too.

peter("Review R/.", tools = extended)
Presets: `minimal` (no ask), `standard`, `readonly` (read and r only; r can still change files, so use `mode = plan` to forbid changes), `extended` (adds grep, find, ls).

peter(q, tools = c("-write", "+grep"))
Remove or add single tools (a bare name adds; an unknown name is an error).

peter("What is in data/scores.csv?")
Reads text and images; binary files get a loading hint.

peter$read("R/fit.R", limit = 40)
peter$grep("lm\\(", path = "R")
peter$find**("*.R"); **peter$ls("R")
Read a file window, search contents, find and list files.

peter$edit("R/fit.R", list(list(
  oldText = "n = 10", newText = "n = 20")))
peter$write("notes.txt", "done\n")
Exact text, else ignoring trailing spaces and quote, dash and space variants (diff printed). Atomic writes.

peter$sh(c("git", "status"))
peter$bg(c("Rscript", "long.R"))
Run a program; run one in the background.

peter$sql("SELECT cyl, AVG(mpg) FROM mtcars
  GROUP BY cyl ORDER BY cyl", mtcars)
SQL on data frames by name (duckdb). `peter$py()` runs Python once you set `RETICULATE_PYTHON` or call `reticulate::py_config()`.

ARTIFACTS: SHINY APPS

peter("Make a Shiny app of it.", survey)
Writes `app.R` in `.gptr/artifacts/<id>/` (temporary before `gptr_init()`) and, at the console, launches it.

gptr_artifacts()
gptr_artifacts("survey-view", open = TRUE)
List apps by id; open, stop or relaunch a version.

Permissions and trust

`plan`	Read only; R runs in a scratch environment. Its plan goes once to your next `peter()` call (option `gptr.plan_handoff`)
`manual`	Default: read-only actions run, others ask; with nobody to ask, the run stops
`edits`	Also the write and edit tools in the project; R code that writes files (`write.csv()`) still asks
`auto`	All but critical actions, gptr settings and secret reads
`yolo`	Nearly all; critical actions still ask

gptr_permissions(deny = "r(sh:rm *)")
Rules: `deny` beats `ask` beats `allow`.

gptr_rewind(s)
Undo the last turn: objects, files and conversation.

The gate is advisory, not a sandbox: `?gptr_security`.

Skills, plugins and MCP

gptr_skills()
Skills from the project, user, plugins and packages.

peter("Speed up f.", f,
  skills = "high-performance-r")
Preload a skill. Write yours as `<name>/SKILL.md` with a `description:` line in `tools::R_user_dir("gptr", "config")/skills`, or in `.gptr/skills/` of a trusted project.

```
.gptr/prompts/check.md  # becomes /check
```
A console command; project templates need trust.

peter("Report units.", plugins = "./lab")
Enable a plugin; one inside the project loads once the project is trusted. `gptr_plugins()` lists them.

gptr_mcp_add("docs",
  url = "http://127.0.0.1:8000/mcp")
Add an MCP server (or `command =` for stdio).

petermcp<server>$<tool>(...)
MCP tools are R functions the agent calls.

```
h = gptr_mcp_serve(); h$config  # paste one
```
h$stop()
Serve this R session to Claude Code, Codex or Cursor. Calls needing approval are refused: set the mode or rules first.

```
up = gptr_tool("shout", "Upper-case text",
  fun = function(x) toupper(x))
off = gptr_register(up)
```
Add a tool; re-registering replaces it, `off()` removes it.

Teams, fan-outs, background

Members inherit the mode. A team or fan-out raises an error only when no member finishes; otherwise `$status` and `$reason` name the members that did not.

```
s = peter("Review fit.", agents = list(
  stats = agent(system = "Check stats."),
  code = agent(system = "Check code.")))
```
A team of named roles; `s$stats$text` reads one.

```
f = peter("Summarise; gptr_return() it.",
  cohorts, parallel = 2)
```
One child per element, two at a time; `f$text` has each answer, `f$value` each `gptr_return()` value.

gptr_parallel(a = **peter**("Plan."),
  b = **peter**("Review."))
Run independent calls concurrently.

```
b = peter("Check.", fit, background = TRUE)
gptr_wait(b)
```
Experimental: return at once; also `gptr_steer()`, `gptr_cancel()`, `gptr_jobs()`.

gptr_usage(s, by = "agent")
Tokens and cost by session, agent, model or route.

Learn more

Guides: broccolito.github.io/gptr/articles (start with Getting started). `browseVignettes("gptr")` lists them only if the install built vignettes (CRAN, `R CMD build`).